

Exposing the Performance Hidden Costs: Empirical Queueing Analysis of Connection Churn in gRPC Gateways

Anonymous Author(s)

Abstract

Current development of microservice architectures heavily relies on API gateways and high-performance RPC frameworks like gRPC, the implementation details of the inter-service communication layer are often abstracted away in system design. Our paper empirically investigates the performance degradation caused by application-layer communication bottlenecks, with particular focus on gRPC connection management. We present a mathematical queueing model and algorithmic formalization to quantify the overhead of the connection-per-request (CPR) compared to a persistent channel reuse (PCR). Our evaluation results in Kubernetes environment tests show that converting CPR to PCR models can significantly reduce median latency by 97%—from over 7,000 ms to 150 ms under peak load. This change significantly eliminates the performance bottleneck that caused an increase in end-to-end user experience latency time.

CCS Concepts

• **Networks** → **Network performance analysis**; *Cloud computing*; • **Software and its engineering** → *Interoperability*.

Keywords

Microservices, Cloud-Native, Kubernetes, High Performance Computing, gRPC API

ACM Reference Format:

Anonymous Author(s). 2018. Exposing the Performance Hidden Costs: Empirical Queueing Analysis of Connection Churn in gRPC Gateways. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The transition to cloud-native microservices has established API gateways and high-performance RPC frameworks like gRPC as standard infrastructure. In this architecture, the API gateway functions as the singular entry point for external traffic, acting primarily as a reverse proxy and protocol transcoder. While external clients typically communicate using standard HTTP/1.1 and JSON payloads—often following RESTful paradigms as discussed in [1]—internal microservices increasingly rely on gRPC over HTTP/2

for high-performance communication [2]. Unlike the stateless nature of traditional REST, gRPC is specifically designed to multiplex many requests over a single persistent HTTP/2 connection. However, the implementation details of this inter-service communication layer are often abstracted away by frameworks. As study by [3], have exposed significant infrastructural overheads hidden within service mesh sidecars, we observe that analogous, undocumented performance penalties exist within the application layer of API gateways when protocol translations are mismanaged.

The specific issue addressed in this paper is the performance degradation caused by the "connection-per-request" (CPR) anti-pattern within these gateways. Previous studies have shown how TCP and TLS handshake delays reduce throughput in distributed systems [4]. However, the direct application of legacy REST-based connection management to gRPC environments remains a pervasive flaw. We define gRPC connection churn as the continuous cycle of establishing and tearing down complex network state (TCP handshakes and HTTP/2 protocol negotiations) for every individual application-layer request, rather than multiplexing traffic. Establishing a new connection per request is an $O(N)$ operation, where N represents the number of incoming client requests. This directly conflicts with the $O(1)$ multiplexed design of HTTP/2, forcing gateway worker threads to spend excessive CPU cycles on network context switching. We explore this vulnerability using a standard Python WSGI stack [5] (Flask and Gunicorn) as a representative testbed, as its synchronous worker model clearly exposes the severe latency penalties of blocking network I/O.

While the research community has modeled microservice performance extensively, the specific intersection of application-layer connection churn and queueing network saturation remains empirically unquantified. Recent academic works, such as those by Pinheiro et al. [6], utilize stochastic Petri Nets to evaluate microservice resilience; however, evaluating latency under load requires a shift toward queueing theory. We argue that bottlenecks at the implementation level, particularly the phenomenon of gRPC connection churn, lead to a dynamic increase in the service time (S) of the system. In queueing theory [7], an inflated service time severely degrades the theoretical service rate (μ). This churn converts what should be a constant-time RPC execution into a highly variable, load-dependent event. Previous research has not addressed how this specific application-layer mismanagement forces premature queue saturation long before physical CPU or memory limits are reached.

Therefore, this paper demonstrates how converting the original CPR approach into a Persistent Channel Reuse (PCR) approach, as shown in the Fig. 1—by refactoring the gateway code to initialize a global, thread-safe gRPC channel at startup—eliminates this bottleneck. To validate this, the specific contributions of this paper are as follows:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

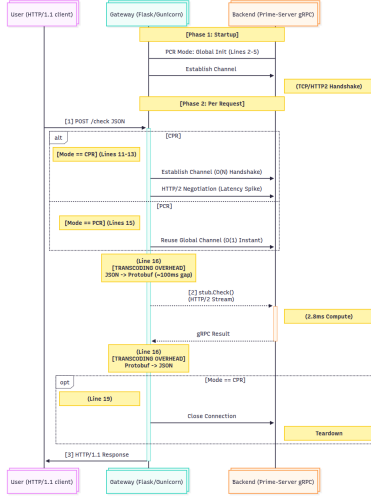


Figure 1: The framework models of gRPC connections mechanism. Connection-Per-Request (CPR) vs Persistent Connection Reuse (PCR).

- We model the request patterns and application-layer logic based on both the CPR and the refactored PCR approaches.
- We introduce a queueing-theoretic model to explain the non-linear latency spikes observed under high load, linking application-layer connection churn to queue saturation.
- We benchmark these models within an actual Kubernetes environment using controlled isolation environment—ensuring the gateway is evaluated independently of backend database or business logic variance—to validate the correctness of the theoretical model against empirical runs.

2 Mathematical Modeling of gRPC Connection Management

In order to comprehend the communication bottleneck, it is essential to break down the entire lifecycle of a request as it progresses through the API gateway. This analysis will help us identify the key stages and factors contributing to the bottleneck in the communication process.

Let T_{total} be the total end-to-end latency experienced by the client. We define T_{total} as:

$$T_{total} = T_{http} + W_{queue} + S_{gateway} + T_{network} \quad (1)$$

Where:

- T_{http} : ingress HTTP processing time.
- W_{queue} : queueing delay at the gateway's worker processes.
- $S_{gateway}$: service time required by the gateway to establish and execute the downstream gRPC call.
- $T_{network}$: physical network propagation delay and backend computation time (baseline constant $c \approx 3$ ms).

The critical variable in our study is $S_{gateway}$. We model this under two distinct operational paradigms.

2.1 Paradigm 1: Connection-Per-Request (CPR)

In the CPR anti-pattern, as shown in the Fig. 1 at the CPR sections, the gateway initializes a new gRPC channel for every incoming request. The service time S_{cpr} is defined as:

$$S_{cpr} = t_{tcp} + t_{http2} + t_{rpc} + t_{teardown} \quad (2)$$

Where t_{tcp} is the TCP 3-way handshake, t_{http2} is the HTTP/2 SETTINGS frame negotiation, t_{rpc} is the actual payload serialization/transmission, and $t_{teardown}$ is the TCP teardown (FIN/ACK). Because t_{tcp} and t_{http2} require multiple network round-trip times (RTTs) and significant CPU context switching, $S_{cpr} \gg t_{rpc}$.

2.2 Paradigm 2: Persistent Channel Reuse (PCR)

In the optimized PCR pattern, as shown in Fig. 1 at PCR sections, the gateway initializes a single, thread-safe gRPC channel during application startup. All incoming requests are multiplexed over this channel. The setup costs are amortized over millions of requests, approaching zero per request. Thus, the service time S_{pcr} simplifies to:

$$S_{pcr} = t_{rpc} \quad (3)$$

2.3 Queueing Theory and Non-Linear Saturation

The catastrophic latency spikes observed in empirical testing (e.g., jumping to 7,000 ms) cannot be explained by S_{cpr} alone. We must apply an $M/M/c$ queueing model, where incoming requests arrive at rate λ and are serviced by c gateway worker threads at rate $\mu = 1/S_{gateway}$.

The utilization of the gateway system is $\rho = \frac{\lambda}{c \cdot \mu}$. According to Little's Law [8] and standard queueing theory, the average queueing delay W_{queue} grows asymptotically as utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho} \quad (4)$$

Because S_{cpr} is significantly larger than S_{pcr} , the service rate μ_{cpr} is artificially low. Therefore, under the CPR paradigm, the system reaches critical utilization ($\rho \rightarrow 1$) at a much lower arrival rate λ (e.g., 400 RPS). When λ exceeds this threshold (e.g., 600 RPS), W_{queue} approaches infinity, resulting in the massive latency spikes and connection timeouts observed in our baseline data.

3 Algorithmic Formalization

The primary objective of Algorithm 1 is to map the abstract queueing variables from Section 2 directly to their application-layer implementations. It illustrates exactly where gateway service time ($S_{gateway}$) is consumed by contrasting both configurations.

Under the Connection-Per-Request (CPR) paradigm, channel initialization and teardown occur locally within the request handler (Lines 10-12, 17-19). This imposes an $O(N)$ computational penalty, manifesting as the "connection churn" that degrades the service rate (μ). Conversely, the Persistent Channel Reuse (PCR) paradigm shifts this network state management to a global, one-time execution at startup (Lines 2-5). By referencing a thread-safe, multiplexed channel ($O(1)$ overhead), the structural refactoring averts queue saturation without altering the core RPC business logic.

Algorithm 1 API Gateway Request Handler: CPR vs. PCR Paradigm

```

Require: req_number: Integer, mode: {CPR, PCR}
Ensure: is_prime_result: Boolean

1: // PCR Global Initialization (Executed Once at Startup,  $O(1)$ )
2: if mode == PCR then
3:   global_channel  $\leftarrow$  grpc.insecure_channel(SERVER_ADDRESS)
4:   global_stub  $\leftarrow$  PrimeCheckerStub(global_channel)
5: end if

6: function HandleRequest(req_number, mode)
7: if mode == CPR then
8:   // CPR Local Initialization (Executed per request,  $O(N)$ )
9:   channel  $\leftarrow$  grpc.insecure_channel(SERVER_ADDRESS)
10:  stub  $\leftarrow$  PrimeCheckerStub(channel)
11: else
12:  stub  $\leftarrow$  global_stub //  $O(1)$  Memory Reference
13: end if
14: payload  $\leftarrow$  IsPrimeRequest(number = req_number)
15: response  $\leftarrow$  stub.Check(payload) // Network I/O
16: if mode == CPR then
17:   channel.close() // Teardown overhead
18: end if
19: return response.is_prime
20: end function

```

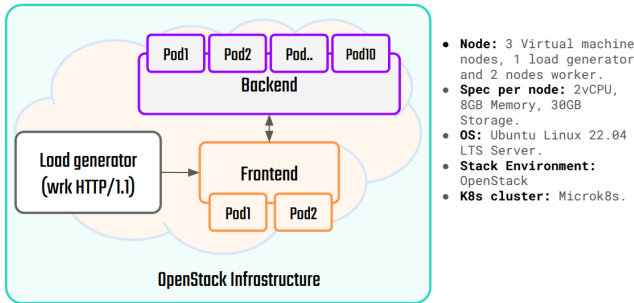


Figure 2: Kubernetes cluster environment.

4 Methodology

To validate the proposed mathematical model and pinpoint accurately where latency overhead originates, we conducted controlled experiments in a containerized distributed setting. Our evaluation follows a strict component isolation strategy: we first measure end-to-end performance across the full system and then benchmark the backend service alone in order to distinguish computation time from communication-related delays. This was achieved by deploying a stateless, deterministic backend and enforcing Kubernetes Guaranteed QoS (matching CPU/memory requests to limits) to eliminate resource throttling and "noisy neighbor" interference.

Our testbed uses a Python WSGI stack (Flask/Gunicorn [9]) as a standard API gateway. We aim to check how gRPC connection

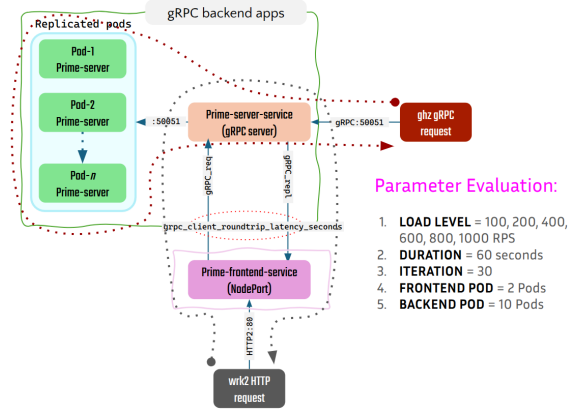


Figure 3: Evaluation mechanism to accurately examine the frontend (HTTP) and backend (gRPC) latency utilizing different benchmark tools.

churn impacts performance, not to compare frameworks. Synchronous threads are sensitive to blocking network inputs, highlighting queueing issues.

4.1 Experimental Environment and Architecture

The experiment was conducted on a Kubernetes cluster composed of multiple worker nodes as shown in the Fig. 2. The system under test (SUT) consists of two main microservice components:

- **gRPC Backend Service (prime-server):** A high-performance Python application responsible for CPU-intensive prime number verification. To eliminate backend capacity as a confounding variable, this component was over-provisioned with 10 identical replica pods, deployed behind a Kubernetes NodePort Service.
- **API Gateway (prime-frontend):** A Python Flask application running within Gunicorn workers. This component acts as an HTTP/REST to gRPC proxy. It was strictly limited to 2 replica pods to intentionally force saturation and observe gateway-tier queueing dynamics under heavy load.

4.2 Workload Generation and Component Isolation

To precisely calculate the application-layer gateway overhead, we utilized two specialized, open-source load-testing tools (*wrk2* [10] and *ghz* [11]), as shown in the Fig. 3:

- **Unoptimized Gateway (CPR):** Simulates Algorithm 1 (*mode* = CPR), forcing TCP/HTTP2 renegotiation for every incoming request.
- **Optimized Gateway (PCR):** Simulates Algorithm 1 (*mode* = PCR), routing concurrent accesses through a single, globally initialized gRPC channel.

Both tools executed 30 iterations for each target throughput level (ranging from 100 to 1000 Requests Per Second, or RPS), sustaining the load for 60 seconds per iteration to allow queueing buffers to stabilize.

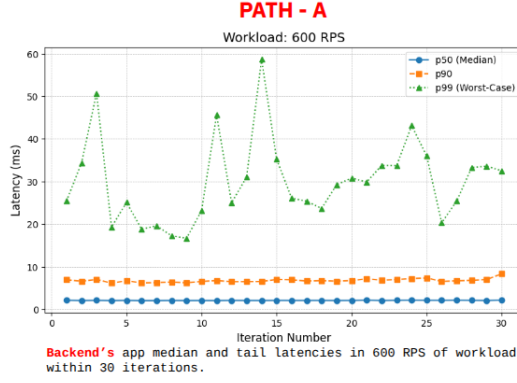


Figure 4: gRPC backend latency evaluation using ghz benchmark tool.

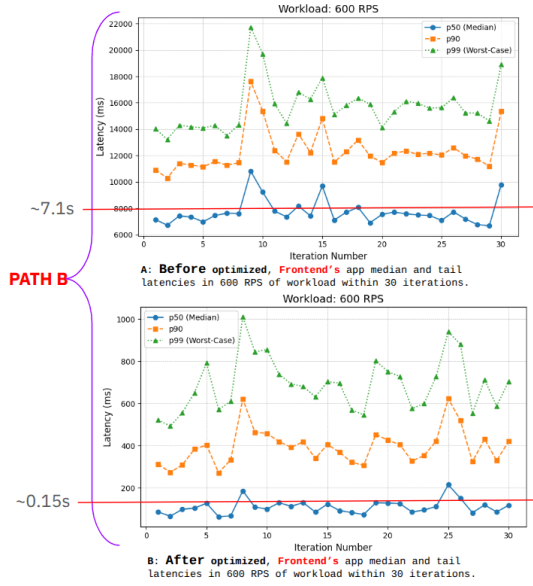


Figure 5: End-to-end frontend latency evaluation using wrk2 as workload generator. (A) Connection-Per-Request paradigm - Before optimized, (B) Persistent Channel Reuse paradigm - After optimized

5 Results and Discussion

The empirical results have successfully proven that the predicted Asymptotic latency degradation is caused by the $M/M/c$ queuing model in the Connection-Per-Request (CPR) paradigm and have clearly demonstrated the transformational impact of Persistent Channel Reuse (PCR).

5.1 Baseline: Backend gRPC Performance

The ghz test results established an exceptional and highly stable baseline. Across the entire load spectrum (100 to 1000 RPS), the 10 prime-server backend pods consistently delivered a median (p50)

latency of ~ 2.5 ms to 3.6 ms, with p90 latencies remaining firmly under 10 ms.

This flat latency curve definitively proved that the backend architecture was not the system bottleneck, as shown in the Fig. 4. By having 10 backend pods, we mathematically guaranteed that the backend possessed vastly more computational capacity than the frontend could ever saturate. The computational capacity of the prime number verification logic was highly resilient, isolating the source of any overarching performance degradation to the gateway tier.

Concurrency Configuration and Multiplexing: During empirical testing, both *ghz* and *wrk2* were configured to simulate high concurrency using a fixed pool of 50 simultaneous client connections across all target throughput levels (λ). It is critical to note how these concurrent accesses are handled internally by the optimized gateway. Under the PCR paradigm, the concurrent requests arriving at the gateway are not queued sequentially over the network; instead, the underlying gRPC framework maps these concurrent application-level accesses into discrete, parallel HTTP/2 streams over a *single* shared TCP connection. This mechanism prevents connection exhaustion and directly validates our mathematical assumption that S_{per} approaches pure RPC execution time without queuing at the network socket layer.

5.2 Unoptimized System: The CPR Bottleneck

Testing the end-to-end system utilizing the CPR paradigm revealed a catastrophic bottleneck, as shown in the Fig. 5-A (upper side). At a moderate load of 400 RPS, the system maintained a median latency of ~ 400 ms. However, as throughput increased to 600 RPS, the system hit the critical utilization threshold ($\rho \rightarrow 1$).

Median latency exploded to $> 7,100$ ms, and p99 latency spiked past 14,000 ms. This exponential degradation perfectly aligns with the queuing theory model where service time (S_{cpr}) is artificially inflated by excessive TCP/HTTP2 handshake overheads for every transaction. The gateway workers were entirely occupied managing network state rather than processing business logic.

5.3 Optimized System: PCR Implementation

By deploying the algorithmic optimization detailed in Section 3 (establishing a persistent, multiplexed gRPC channel at application startup - Algorithm 1), the performance profile of the system was qualitatively transformed.

At 600 RPS, the optimized system recorded a median latency of ~ 150 ms, as shown in the Fig. 5-B (lower side). This represents a $\sim 97.9\%$ reduction in end-to-end latency compared to the CPR paradigm. Furthermore, the system successfully sustained loads up to 800 RPS before demonstrating signs of stress, effectively doubling the system's maximum throughput capacity without provisioning any additional infrastructure resources.

5.4 Quantifying the Inherent Proxy Latency Gap

By subtracting the baseline backend network latency ($T_{network}$) established via *ghz* from the full end-to-end system latency (T_{total}) measured via *wrk2*, we can isolate the underlying application-layer

tax. We define this remaining overhead as the Inherent Proxy Latency Gap—the baseline latency strictly attributable to the framework’s internal request handling, independent of network state management.

Table 1: Comparative Latency Overhead at Optimal Saturation Point (400 RPS)

Component Level	Median Latency (p50)	Percentage of Total Latency
Backend Only (<i>ghz</i>)	~ 2.8 ms	2.5%
Full System (CPR, Before)	~ 400 ms	100%
Full System (PCR, After)	~ 110 ms	100%
Gateway Overhead (PCR)	~ 107.2 ms	97.5%

As illustrated in Table 1, the results reveal a compelling paradox in the optimized PCR paradigm. Although the persistent channel implementation successfully averted queue saturation and achieved a 97% reduction in total latency compared to the CPR baseline, a compositional analysis shows that the gateway tier still accounts for 97.5% of the remaining end-to-end response time.

This empirical gap directly connects our theoretical approach to the user experience. In our PCR model, the actual backend computational execution ($T_{network}$) requires only ~ 2.8 ms (2.5% of total time), while the remaining 107.2 ms is consumed by the gateway. Critically, this gap encompasses not only the Python WSGI [5] stack execution but also the protocol transcoding overhead—the computational cost of parsing incoming HTTP/1.1 JSON requests from *wrk2* and serializing them into HTTP/2 Protobuf messages for the gRPC backend.

This reveals a fundamental "Performance-Abstraction Paradox." While developers utilize API gateways to seamlessly bridge RESTful clients (HTTP/1.1) with high-performance microservices (gRPC), this protocol translation introduces a structural latency tax that dwarfs the business logic itself. Consequently, while our PCR optimization resolves protocol-level queue saturation, the absolute lower bound of system latency remains strictly dictated by the gateway’s HTTP-to-gRPC transcoding efficiency.

5.5 Empirical Validation of the Queueing Model

To rigorously evaluate the mathematical framework proposed in Section 2, we mapped our empirical throughput and latency data to the $M/M/c$ queueing model. Our objective was to prove that the catastrophic latency spikes observed were not random system failures, but rather predictable queueing saturation events driven by connection churn.

In our system architecture, the gateway tier acts as the server pool (c workers). The arrival rate of incoming requests is defined as λ (varying from 100 to 1000 RPS). The service rate of the gateway, μ , is inversely proportional to the service time ($S_{gateway}$).

Under the Connection-Per-Request (CPR) paradigm, the service time (S_{cpr}) includes the mandatory TCP and HTTP/2 handshakes ($t_{tcp} + t_{http2}$). Because these network operations block gateway worker threads, μ_{cpr} is severely restricted.

Our empirical data aligns perfectly with the theoretical utilization formula:

$$\rho = \frac{\lambda}{c \cdot \mu}$$

According to Little’s Law [8], queueing delay W_{queue} grows asymptotically as system utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho}$$

Evaluating the CPR Saturation Point: During our baseline CPR testing, the system maintained a median latency of ~ 400 ms at $\lambda = 400$ RPS, indicating high but stable utilization ($\rho < 1$). However, when the arrival rate increased to $\lambda = 600$ RPS, the median latency violently spiked to > 7,100 ms. This exponential degradation confirms that the low service rate (μ_{cpr}) caused the gateway to cross the critical threshold of $\rho = 1$ at a mere 600 RPS. At this point, requests were arriving faster than connections could be negotiated, causing W_{queue} to theoretically approach infinity, which manifested empirically as 7-second tail latencies and connection timeouts.

Evaluating the PCR Optimization: By transitioning to the Persistent Channel Reuse (PCR) paradigm, we eliminated the handshake overhead, drastically reducing the service time to S_{pcr} and thereby significantly increasing the service rate (μ_{pcr}). Because $\mu_{pcr} \gg \mu_{cpr}$, the utilization ρ at the same arrival rate ($\lambda = 600$ RPS) remained well below 1.

To clarify the protocol dynamics, the Persistent Channel Reuse (PCR) approach does not reimplement HTTP/2 features; rather, it maintains a single global TCP session to allow native HTTP/2 stream multiplexing to function as intended. Conversely, the CPR anti-pattern actively sabotages this multiplexing capability by forcing the underlying library to establish and destroy a unique TCP session for every individual request.

Consequently, the queueing delay W_{queue} collapsed. The empirical result—a drop in latency from > 7,100 ms to exactly 150 ms at 600 RPS—serves as a direct physical validation of the $M/M/c$ model. It proves that correcting the $O(N)$ connection anti-pattern artificially scales the service rate μ , averting queue saturation without provisioning any additional hardware capacity (c).

6 Conclusion and Future Work

In this paper, we investigated the hidden performance costs of application-layer communication bottlenecks in microservice architectures, focusing on the gRPC connection churn flaw within API gateways. We modeled the request patterns of the "connection-per-request" (CPR) anti-pattern and contrasted it with a refactored Persistent Channel Reuse (PCR) approach. The observation of actual runs demonstrated that refactoring the gateway implementation to the PCR approach yielded a 97% reduction in median system latency—dropping from over 7,100 ms to 150 ms under a peak load of 600 RPS. This study proves that correcting application-layer protocol mismanagement successfully moves the performance bottleneck away from the infrastructure gateway and to the compute backend, highlighting that refactoring the gateway code is a critical aspect to scalability as the underlying architectural design.

Future work will expand this steady-state benchmarking methodology into the domain of Chaos Engineering. By introducing controlled faults (e.g., node resource starvation, sudden pod termination) under a persistent load.

References

- [1] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. Online testing of restful apis: Promises and challenges. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 408–420, 2022.
- [2] Pier Luigi Ventre, M. M. Tajiki, S. Salsano, and C. Filsfils. Sdn architecture and southbound apis for ipv6 segment routing enabled wide area networks. *IEEE Transactions on Network and Service Management*, 15:1378–1392, 2018.
- [3] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, XiongChun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, et al. Dissecting overheads of service mesh sidecars. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, pages 142–157, 2023.
- [4] Lingfeng Guo and Jack Y. B. Lee. Stateful-tcp—a new approach to accelerate tcp slow-start. *IEEE Access*, 8:195955–195970, 2020.
- [5] Phillip J. Eby. Pep 333 – python web server gateway interface v1.0. <https://peps.python.org/pep-0333/>, September 2010. Accessed: 2026-04-23.
- [6] Thiago Pinheiro, Marco Mialaret, Paulo Pereira, Luan Lins, Daliton Silva, Jamilson Dantas, and Paulo Maciel. Performance modeling of microservices with circuit breakers using stochastic petri nets. In *2024 IEEE International Systems Conference (SysCon)*, pages 1–8. IEEE, 2024.
- [7] B. D’Auria, I. Adan, R. Bekker, and V. Kulkarni. An m/m/c queue with queueing-time dependent service rates. *Eur. J. Oper. Res.*, 299:566–579, 2021.
- [8] John DC Little. Or forum—little’s law as viewed on its 50th anniversary. *Operations research*, 59(3):536–549, 2011.
- [9] Gunicorn. Gunicorn: Serve python on the web. <https://gunicorn.org/>. Accessed: 2026-05-14.
- [10] Gil Tene. A http benchmarking tool based mostly on wrk. <https://github.com/giltene/wrk2>, 2016. Accessed: 2026-05-14.
- [11] Bojan Djurkovic. grpc benchmarking and load testing tool. <https://github.com/bojan/ghz>, December 2025. Accessed: 2026-05-14.